CS6650 HW1b

Sai Karthikeyan, Sura

January 19, 2026

1 Part I: Deployment Evidence

I Deployed the Go-Gin web service to an AWS EC2 instance (t2.micro) running Amazon Linux 2023. Figure 1 shows the SSH session with the server successfully listening on port 8080. Figure 2 confirms the server is accessible via the public IP and returns the expected JSON responses.

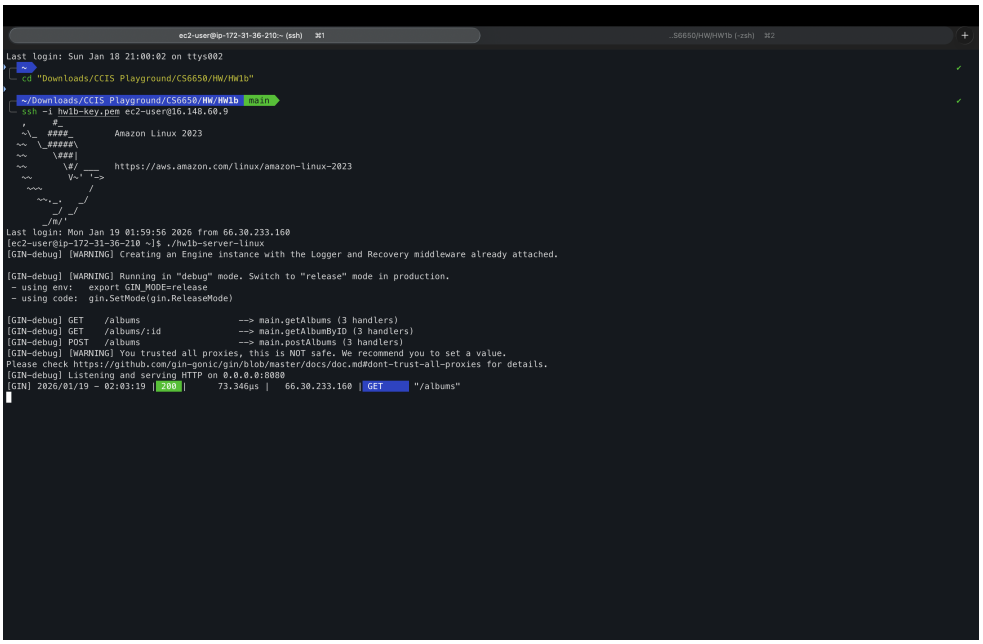

Figure 1: Server startup logs on AWS EC2 instance.

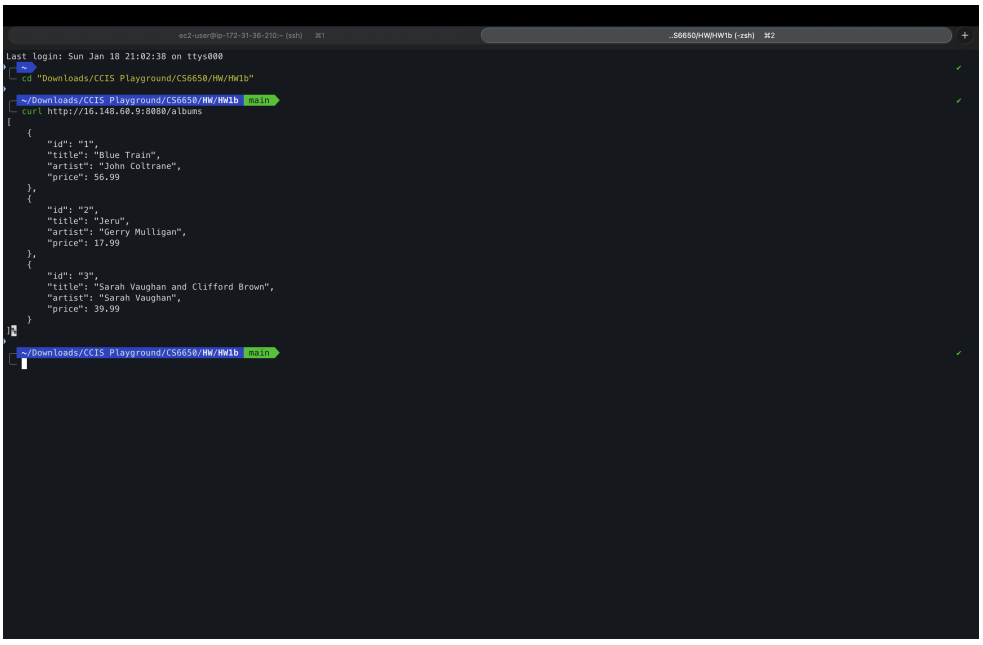

Figure 2: Successful cURL request to AWS EC2 instance.

2 Part II: Learning Outcomes

2.1 Virtual Machines vs. Local Execution

A Virtual Machine (VM) is a software-based emulation of a physical computer. When running the server locally, it executes on my personal hardware (macOS) and is restricted to my local network (localhost). In contrast, running it on an AWS EC2 instance means the code executes on a slice of shared physical hardware in Amazon's data center. The EC2 instance is ephemeral and exposed to the public internet via a public IP, requiring explicit security group (firewall) configurations to allow traffic.

2.2 IP Address Management

By default, an EC2 instance is assigned a dynamic public IP address. If the instance is stopped and restarted, this IP address changes, which would break any client configurations pointing to the old IP. To solve this, we can allocate an **Elastic IP** address. An Elastic IP is a static IPv4 address designed for dynamic cloud computing, allowing us to mask instance failures by rapidly remaping the address to another instance.

2.3 Instance Types

I selected the **t2.micro** instance type for this assignment.

- **Specs:** 1 vCPU, 1 GiB Memory.
- **Why it matters:** This instance falls under the AWS Free Tier. However, it utilizes "CPU Credits" for bursting. If the application sustains high CPU usage (e.g., heavy load testing), it will consume credits and eventually be throttled, leading to performance degradation. Understanding these specs is crucial for cost management and performance planning.

2.4 GCP vs. AWS Experience

Comparing this experience to HW1a on Google Cloud Platform (GCP):

- **Configuration:** AWS EC2 felt more manual (Infrastructure-as-a-Service). I had to explicitly configure the OS, cross-compile the binary for the specific architecture (AMD64), and manually handle SSH keys and Security Groups.
- **GCP:** In previous experiences (like App Engine or Cloud Run), much of the underlying infrastructure (OS patching, networking) is abstracted away (Platform-as-a-Service), making deployment simpler but offering less granular control than a raw EC2 instance.

3 Part III: Performance Testing & Observations

Performed a load test using a Python script to send sequential GET requests to the server for 30 seconds.

3.1 Test Execution Output

Figure 3 shows the terminal output as evidence of execution. It indicates that a total of 121 requests were made, with an average latency of 248.10ms.

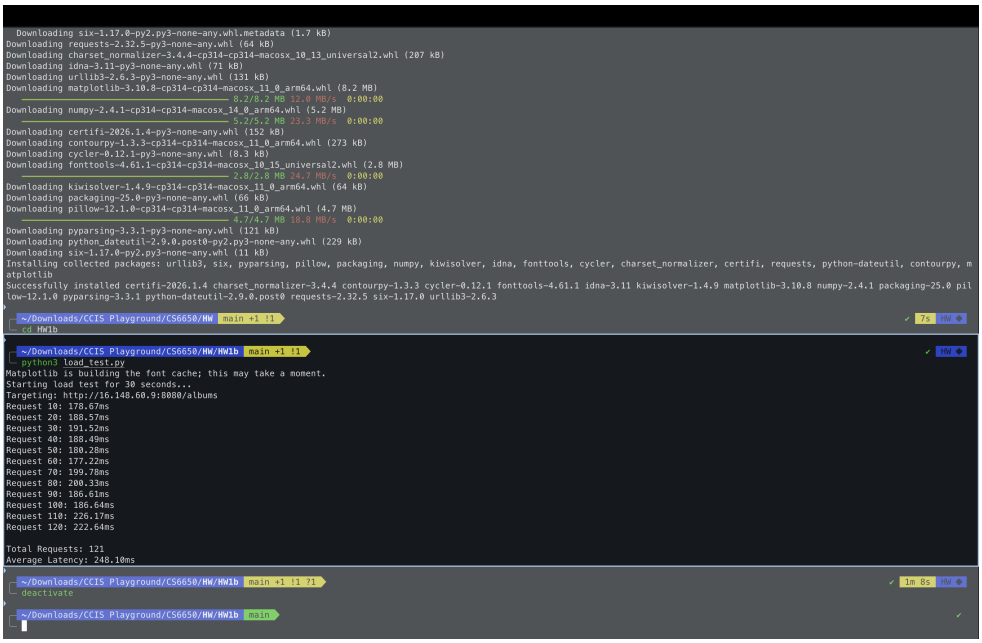

Figure 3: Terminal output showing the Load Test execution and final statistics.

3.2 Visual Results

The script produced the visualization in Figure 4, which illustrates the distribution of response times observed during the load test.

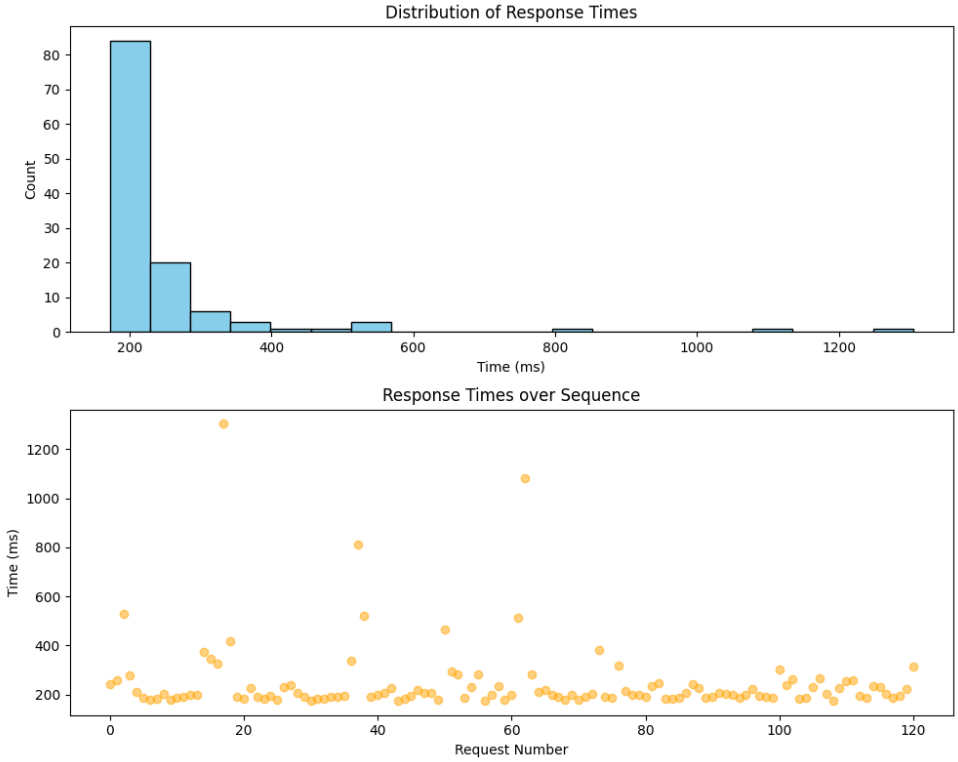

Figure 4: Histogram and Scatter Plot of Response Times.

3.3 Observations

1. **The Long Tail:** As seen in the histogram, while the majority of requests clustered around 180-220ms, there were distinct outliers (the "long tail"). This variance is expected in a cloud environment due to network jitter and the "noisy neighbor" effect on shared hardware.

2. **Infrastructure Impact:** Running on a single `t2.micro` instance introduces variability. Since the instance has limited CPU and memory, even a small spike in processing (or background system tasks) can cause request queuing, leading to higher latency for specific requests.

3. **Network vs. Processing:** The baseline latency ($\sim$180ms) suggests that network round-trip time (RTT) is the dominant factor, given the geographic distance between my client and the AWS Oregon region. The spikes (outliers) are likely due to server-side processing delays or momentary network congestion.

4. **Scaling Implications:** The current setup processes requests sequentially. If we scaled to 100 concurrent users, the single-threaded nature of the test (and potentially the server's resource limits) would create a bottleneck. To handle such load, we would need to scale horizontally (add more instances behind a Load Balancer) or vertically (upgrade to a larger instance type).